

Advanced Numerical Simulation and Statistical Analysis

Using NumPy and Generator Expressions

Dr. M. Mallikaarjunan
SASTRA

Contents

1	Random Sensor Readings and Normalization	2
2	Sum of Squares of Non-zero Matrix Elements	5
3	Monte-Carlo Estimation of π	6
4	Stability of Random 3×3 Matrices	8
5	Sliding Windows and Covariance	9
6	Mean, Variance and Median of 500 Random Values	10
7	Pairs of Highly Correlated Feature Vectors	12
8	Principal Component Analysis (PCA)	14
9	Most Influential Node from a Dominant Eigenvector	17
10	Quick Revision: What Each Program Teaches	20

Program 1. Random Sensor Readings and Normalization

Problem

Generate 1000 random sensor readings using a generator expression, convert them into a NumPy array, and normalize the readings using the mean and standard deviation.

What does normalization mean?

Suppose the observations are

$$x_1, x_2, \dots, x_n.$$

Their population mean and population standard deviation are

$$\mu = \frac{1}{n} \sum_{i=1}^n x_i, \quad \sigma = \sqrt{\frac{1}{n} \sum_{i=1}^n (x_i - \mu)^2}.$$

The **standardized or normalized value** corresponding to x_i is

$$z_i = \frac{x_i - \mu}{\sigma}.$$

This transformation shifts the data so that the new mean is 0 and rescales it so that the new standard deviation is 1.

Why does the normalized mean become zero?

From the definition of the mean, the positive and negative deviations of the observations from their mean balance exactly. Hence

$$\sum_{i=1}^n (x_i - \mu) = 0.$$

Therefore

$$\bar{z} = \frac{1}{n} \sum_{i=1}^n z_i = \frac{1}{n} \sum_{i=1}^n \frac{x_i - \mu}{\sigma} = \frac{1}{n\sigma} \sum_{i=1}^n (x_i - \mu) = \boxed{0}.$$

Thus normalization centers the data exactly at zero (apart from tiny floating-point round-off in computer output).

Similarly,

$$\text{Var}(z) = \frac{1}{n} \sum_{i=1}^n \left(\frac{x_i - \mu}{\sigma} \right)^2 = \frac{1}{\sigma^2} \left[\frac{1}{n} \sum_{i=1}^n (x_i - \mu)^2 \right] = 1,$$

so

$$\boxed{\text{Std}(z) = 1}.$$

Simple numerical example

Take

$$2, 4, 6.$$

Then

$$\mu = \frac{2 + 4 + 6}{3} = 4, \quad \sigma^2 = \frac{(-2)^2 + 0^2 + 2^2}{3} = \frac{8}{3}, \quad \sigma = \sqrt{\frac{8}{3}}.$$

Therefore,

$$(z_1, z_2, z_3) = \left(\frac{-2}{\sqrt{8/3}}, 0, \frac{2}{\sqrt{8/3}} \right) \approx (-1.225, 0, 1.225).$$

Now the mean of the normalized values is

$$\bar{z} = \frac{-1.225 + 0 + 1.225}{3} = 0.$$

To verify the standard deviation, first compute the variance:

$$\text{Var}(z) = \frac{1}{3} [(-1.225)^2 + 0^2 + (1.225)^2].$$

Since

$$(1.225)^2 \approx 1.5006,$$

we obtain

$$\text{Var}(z) \approx \frac{1.5006 + 0 + 1.5006}{3} \approx 1.$$

Hence,

$$\text{Std}(z) = \sqrt{\text{Var}(z)} = \sqrt{1} = 1.$$

Thus,

Normalized Mean = 0, Normalized Standard Deviation = 1

Therefore, normalization transforms the original data

$$(2, 4, 6)$$

into approximately

$$(-1.225, 0, 1.225),$$

whose mean is 0 and population standard deviation is 1.

Python program

```
1 import numpy as np
2
3 # Generate 1000 random numbers
4 data = np.fromiter((np.random.rand() for _ in range(1000)), dtype=float)
5
6 # Compute mean and standard deviation of original data
7 mean = np.mean(data)
8 std = np.std(data)
9
10 # Normalize the data
11 normalized = (data - mean) / std
12
13 # Print results
14 print("Original Mean =", mean)
15 print("Original Standard Deviation =", std)
16 print("Normalized Data =", normalized)
17
18 # Check mean and std of normalized data
19 print("Normalized Mean =", np.mean(normalized))
```

```

20 print("Normalized Standard Deviation =", np.std(normalized))
21 print(np.isclose(np.mean(normalized), 0))      # True
22 print(round(np.mean(normalized), 10))          # 0.0

```

Visual Trace

Generator part

`(np.random.rand() for _ in range(1000))`

produces one random number at a time. The function `np.fromiter(...)` consumes those values and forms a NumPy array.

For an illustrative mini-run with data `[2, 4, 6]`:

$$[2, 4, 6] \xrightarrow{\text{mean}} 4 \xrightarrow{\text{subtract mean}} [-2, 0, 2] \xrightarrow{\div \sqrt{8/3}} [-1.225, 0, 1.225].$$

Finally NumPy checks

$$\text{mean}(z) \approx 0, \quad \text{std}(z) \approx 1.$$

Program 2. Sum of Squares of Non-zero Matrix Elements

Problem

Generate a 50×50 integer matrix. Using a generator expression, compute the sum of squares of all non-zero elements.

Mathematical idea

If $A = [a_{ij}]$ is a matrix, the required quantity is

$$E = \sum_{a_{ij} \neq 0} a_{ij}^2.$$

Squaring makes every contribution non-negative. The code first changes the matrix into a one-dimensional sequence with `flatten()`, filters out zero entries, squares the remaining entries, and adds them.

Simple numerical example

Take

$$A = \begin{bmatrix} 2 & 0 & -1 \\ 0 & 3 & 0 \end{bmatrix}.$$

Flattening gives

$$[2, 0, -1, 0, 3, 0].$$

The non-zero values are 2, -1, 3, hence

$$E = 2^2 + (-1)^2 + 3^2 = 4 + 1 + 9 = \boxed{14}.$$

Python program

```
1 import numpy as np
2
3 A = np.random.randint(-5, 5, (50, 50))
4 energy = sum(x**2 for x in A.flatten() if x != 0)
5 print("Energy:", energy)
```

Visual Trace

The generator expression is

`x**2 for x in A.flatten() if x != 0`

and should be read from left to right as

square x $\leftarrow$ for each x in flattened A $\leftarrow$ only if $x \neq 0$.

For the small matrix above:

$$A \rightarrow [2, 0, -1, 0, 3, 0] \rightarrow [2, -1, 3] \rightarrow [4, 1, 9] \rightarrow 14.$$

Important Note

`np.random.randint(-5,5,...)` generates integers from -5 up to 4 ; the upper endpoint 5 is excluded.

Program 3. Monte-Carlo Estimation of π

Problem

Generate random points and estimate π by checking whether each point lies inside a unit circle.

Geometric probability behind the method

The supplied code generates

$$0 \leq x < 1, \quad 0 \leq y < 1,$$

so the random points lie in the unit square $[0, 1] \times [0, 1]$. Inside that square, the condition

$$x^2 + y^2 \leq 1$$

describes the **quarter** of the unit circle in the first quadrant.

The square has area 1, while the quarter-circle has area $\pi/4$. Therefore

$$\frac{N_{\text{inside}}}{N} \approx \frac{\pi/4}{1} = \frac{\pi}{4},$$

and hence

$$\pi \approx 4 \frac{N_{\text{inside}}}{N}.$$

Visualization of the geometry

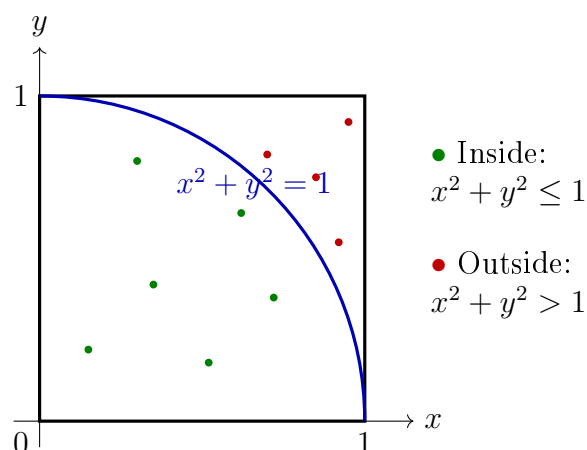


Figure: The actual supplied code samples the first-quadrant unit square and counts points falling in the quarter-circle.

Relation with the full square $[-1, 1] \times [-1, 1]$

The same idea is often drawn with the entire unit circle inside

$$[-1, 1] \times [-1, 1].$$

That square has area 4 and the circle has area π , so again

$$\frac{N_{\text{inside}}}{N} \approx \frac{\pi}{4} \implies \pi \approx 4 \frac{N_{\text{inside}}}{N}.$$

To sample that full square, one would use $x = 2U - 1$ and $y = 2V - 1$ with $U, V \sim U(0, 1)$. The supplied program, however, uses the equivalent first-quadrant version.

Simple numerical example

Suppose $N = 20$ random points are generated and 16 are inside. Then

$$\pi \approx 4 \frac{16}{20} = 3.2.$$

If N becomes very large, the estimate generally approaches the true value 3.14159....

Python program

```
1 import numpy as np
2 N = 10000
3
4 points = ((np.random.rand(), np.random.rand()) for _ in range(N))
5
6 inside = sum(1 for x, y in points if x*x + y*y <= 1)
7
8 pi_est = 4 * inside / N
9
10 print("Estimated Pi:", pi_est)
```

Visual Trace

For each generated point (x, y) , Python checks

$$d = x^2 + y^2.$$

- If $d \leq 1$, add 1 to `inside`.
- If $d > 1$, add nothing.

For example:

$$(0.2, 0.3) : \quad 0.2^2 + 0.3^2 = 0.13 \leq 1 \quad \Rightarrow \quad \text{inside,}$$

$$(0.9, 0.8) : \quad 0.9^2 + 0.8^2 = 1.45 > 1 \quad \Rightarrow \quad \text{outside.}$$

After 10,000 repetitions,

$$\text{pi_est} = 4 \times \frac{\text{inside}}{10000}.$$

Program 4. Stability of Random 3×3 Matrices

Problem

Generate 10 random 3×3 matrices and retain the matrices for which all eigenvalues have magnitude less than 1.

Why eigenvalues determine discrete-time stability

Consider the discrete linear system

$$x_{k+1} = Ax_k.$$

Repeated substitution gives

$$x_k = A^k x_0.$$

If every eigenvalue λ of A satisfies

$$|\lambda| < 1,$$

then the corresponding modes decay geometrically as k increases. Such a matrix is called **Schur stable**.

Simple numerical example

Take

$$A = \begin{bmatrix} 0.5 & 0 \\ 0 & -0.2 \end{bmatrix}.$$

Its eigenvalues are 0.5 and -0.2 , so

$$|0.5| < 1, \quad |-0.2| < 1.$$

Therefore the system is stable.

But for

$$B = \begin{bmatrix} 1.2 & 0 \\ 0 & 0.4 \end{bmatrix},$$

one eigenvalue has magnitude $1.2 > 1$, so B is unstable.

Python program

```
1 import numpy as np
2
3 # Generate 10 random 3x3 matrices
4 matrices = [np.random.randn(3, 3) for _ in range(10)]
5
6 # Select only stable matrices
7 stable = (A for A in matrices if np.all(np.abs(np.linalg.eigvals(A)) < 1))
8
9 # Print each stable matrix
10 for s in stable:
11     print("Stable Matrix:\n", s)
```


Visual Trace

For every matrix A :

$$A \xrightarrow{\text{eigvals}} (\lambda_1, \lambda_2, \lambda_3) \xrightarrow{\text{abs}} (|\lambda_1|, |\lambda_2|, |\lambda_3|) \xrightarrow{\leq 1} (\text{True/False}) \xrightarrow{\text{np.all}} \text{True/False}.$$

Only when all three tests are True is the matrix yielded by the generator.

Important Note

Because `np.random.randn(3,3)` produces arbitrary normally distributed entries, it is possible that no matrix among the 10 samples is stable. In that case nothing is printed after the loop; this is not a program error.

Program 5. Sliding Windows and Covariance

Problem

Generate a time series of 200 values, move a window of length 10 through it, and compute `np.cov(w)` for each window.

Sliding window

For a sequence

$$x_0, x_1, \dots, x_{199},$$

a window of size 10 produces

$$W_0 = (x_0, \dots, x_9), \quad W_1 = (x_1, \dots, x_{10}), \quad \dots, \quad W_{190} = (x_{190}, \dots, x_{199}).$$

Hence the number of windows is

$$200 - 10 + 1 = \boxed{191}.$$

What is covariance?

For two variables X and Y observed n times, sample covariance is

$$\text{Cov}(X, Y) = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y}).$$

A covariance matrix for several variables stores all variances and pairwise covariances:

$$C = \begin{bmatrix} \text{Var}(X_1) & \text{Cov}(X_1, X_2) & \cdots \\ \text{Cov}(X_2, X_1) & \text{Var}(X_2) & \cdots \\ \vdots & \vdots & \ddots \end{bmatrix}.$$

Important point about the supplied code

Each `w` in the supplied program is only a **one-dimensional** array of 10 values. Therefore `np.cov(w)` returns a single sample variance (a 0-dimensional NumPy result), not a multi-variable covariance matrix. A genuine covariance matrix requires two or more variables

per window. The code below is preserved exactly because it is the requested Lab-9 program.

Simple numerical example

For the short sequence

$$[2, 4, 6, 8, 10]$$

and window size 3, the windows are

$$[2, 4, 6], \quad [4, 6, 8], \quad [6, 8, 10].$$

For the first window, mean = 4 and the sample variance is

$$\frac{(2-4)^2 + (4-4)^2 + (6-4)^2}{3-1} = \frac{8}{2} = 4.$$

Thus `np.cov([2,4,6])` gives 4.

Python program

```
1 import numpy as np
2
3 data = np.random.rand(200)
4
5 windows = (data[i:i+10] for i in range(len(data)-9))
6
7 covariances = [np.cov(w) for w in windows]
8
9 print(covariances)
```

Visual Trace

The index i runs from 0 to 190:

i	window
0	data[0:10]
1	data[1:11]
2	data[2:12]
$\vdots$	$\vdots$
190	data[190:200]

Therefore exactly 191 covariance/variance values are produced.

Program 6. Mean, Variance and Median of 500 Random Values

Problem

Generate 500 random numbers using a generator expression and calculate the mean, variance and median.

Three measures of a dataset

For data $x_1, \dots, x_n$:

$$\text{Mean} = \mu = \frac{1}{n} \sum_{i=1}^n x_i$$

measures the arithmetic center,

$$\text{Variance} = \frac{1}{n} \sum_{i=1}^n (x_i - \mu)^2$$

measures spread around the mean, and the **median** is the middle value after sorting the observations.

Simple numerical example

Take

1, 2, 3, 4, 10.

Then

$$\text{Mean} = \frac{20}{5} = 4, \quad \text{Median} = 3.$$

The population variance is

$$\frac{(1-4)^2 + (2-4)^2 + (3-4)^2 + (4-4)^2 + (10-4)^2}{5} = \frac{50}{5} = 10.$$

Notice that the large value 10 pulls the mean upward, while the median remains 3.

Python program

```
1 import numpy as np
2
3 # Generate 500 random numbers using a generator expression
4 data = np.fromiter((np.random.rand() for _ in range(500)), dtype=float)
5
6 # Compute mean, variance, and median
7 print("Mean:", np.mean(data))
8 print("Variance:", np.var(data))
9 print("Median:", np.median(data))
```

Visual Trace

Generator of 500 values $\xrightarrow{\text{np.fromiter}}$ NumPy array

then NumPy independently calculates

`np.mean(data)`, `np.var(data)`, `np.median(data)`.

Since the values come from the uniform distribution on $[0, 1)$, for a large sample we usually expect the mean and median to be near 0.5, while the variance should be near $1/12 \approx 0.0833$; exact output changes each run.

Program 7. Pairs of Highly Correlated Feature Vectors

Problem

Generate 20 random feature vectors of dimension 5 and identify all pairs whose correlation coefficient is greater than 0.9.

Correlation coefficient

For two vectors

$$X = (x_1, \dots, x_n), \quad Y = (y_1, \dots, y_n),$$

the Pearson correlation coefficient is

$$r = \frac{\sum_{k=1}^n (x_k - \bar{x})(y_k - \bar{y})}{\sqrt{\sum_{k=1}^n (x_k - \bar{x})^2} \sqrt{\sum_{k=1}^n (y_k - \bar{y})^2}}.$$

Its value lies between -1 and 1 .

r value	meaning
$r \approx 1$	strong positive linear relationship
$r \approx 0$	weak/no linear relationship
$r \approx -1$	strong negative linear relationship

The program selects pairs with

$$r > 0.9.$$

Why do we import combinations?

The statement

```
1 from itertools import combinations
```

imports the function `combinations` from Python's built-in `itertools` module. We need it because the program must compare **every distinct pair** of the 20 vectors, but it must not compare a vector with itself and must not repeat the same pair in reverse order.

For a small set of four indices

0, 1, 2, 3,

`combinations(range(4), 2)` produces

(0, 1), (0, 2), (0, 3), (1, 2), (1, 3), (2, 3).

It does **not** produce (0, 0), (1, 1), etc., and after producing (0, 1) it does not produce (1, 0). Thus each unordered pair appears exactly once.

For 20 vectors, the number of different pairs is

$$\binom{20}{2} = 190.$$

Therefore the line

```
1 for i, j in combinations(range(20), 2)
```

means: “take the vector indices two at a time, without repetition, and test the correlation of each pair.” This is much shorter and safer than writing nested loops and manually avoiding duplicate pairs.

Simple numerical example

Consider

$$X = (1, 2, 3, 4, 5), \quad Y = (2, 4, 6, 8, 10).$$

Here $Y = 2X$, so they move perfectly together and

$$r = 1.$$

Thus this pair satisfies $r > 0.9$.

Why $[0, 1]$?

`np.corrcoef(X,Y)` returns the matrix

$$\begin{bmatrix} 1 & r \\ r & 1 \end{bmatrix}.$$

Therefore the entry in row 0, column 1 is exactly the required correlation r . Since

$$\begin{bmatrix} \text{corr}(a, a) & \text{corr}(a, b) \\ \text{corr}(b, a) & \text{corr}(b, b) \end{bmatrix}$$

Python program

```
1 import numpy as np
2 from itertools import combinations
3
4 vectors = np.random.rand(20, 5)
5
6 pairs = ((i, j) for i, j in combinations(range(20), 2)
7         if np.corrcoef(vectors[i], vectors[j])[0, 1] > 0.9)
8
9 print(list(pairs))
```

Visual Trace

There are

$$\binom{20}{2} = \frac{20 \cdot 19}{2} = \boxed{190}$$

different unordered pairs. `combinations(range(20), 2)` generates

$$(0, 1), (0, 2), \dots, (0, 19), (1, 2), \dots, (18, 19).$$

For each pair (i, j) :

$$(v_i, v_j) \xrightarrow{\text{np.corrcoef}} r_{ij} \xrightarrow{r_{ij} > 0.9} \begin{cases} \text{keep } (i, j), & \text{True,} \\ \text{discard,} & \text{False.} \end{cases}$$

Important Note

Because the 20 vectors are independent random vectors, sometimes the final list can be empty. An empty list means that no pair happened to have correlation above 0.9 in that particular run.

Program 8. Principal Component Analysis (PCA)

Problem

Create a 100×5 dataset, calculate its covariance matrix, determine the first principal component, and project the centered data onto it.

What does PCA do?

Principal Component Analysis (PCA) is a mathematical technique used to reduce the number of features in a dataset while keeping the most important information. It transforms correlated variables into a smaller set of uncorrelated variables called principal components.

Principal Component Analysis (PCA) replaces the original correlated features by new mutually perpendicular directions called **principal components**. The first principal component is chosen so that the projected data have the largest possible variance. Hence it keeps the strongest pattern or spread present in the data.

For the Lab-9 matrix X with 100 observations and 5 features,

$$X \in \mathbb{R}^{100 \times 5}.$$

PCA reduces each five-dimensional observation to one number when only the first principal component is retained.

A complete simple numerical example

Before using the 100×5 random dataset, consider only three observations with two features:

$$X = \begin{bmatrix} 2 & 1 \\ 4 & 3 \\ 6 & 5 \end{bmatrix}.$$

Each row is one observation and each column is one feature.

Step 1: Find the mean of each feature

The column means are

$$\bar{x}_1 = \frac{2 + 4 + 6}{3} = 4, \quad \bar{x}_2 = \frac{1 + 3 + 5}{3} = 3.$$

Thus the mean vector is

$$\bar{X} = (4, 3).$$

Step 2: Center the observations

Subtract the corresponding column mean from every observation:

$$X_c = \begin{bmatrix} 2-4 & 1-3 \\ 4-4 & 3-3 \\ 6-4 & 5-3 \end{bmatrix} = \begin{bmatrix} -2 & -2 \\ 0 & 0 \\ 2 & 2 \end{bmatrix}.$$

Now each centered feature has mean zero.

Step 3: Form the covariance matrix

NumPy's default sample covariance divides by $n - 1$. Since $n = 3$,

$$C = \frac{1}{3-1} X_c^T X_c.$$

Now

$$X_c^T X_c = \begin{bmatrix} -2 & 0 & 2 \\ -2 & 0 & 2 \end{bmatrix} \begin{bmatrix} -2 & -2 \\ 0 & 0 \\ 2 & 2 \end{bmatrix} = \begin{bmatrix} 8 & 8 \\ 8 & 8 \end{bmatrix}.$$

Therefore

$$C = \begin{bmatrix} 4 & 4 \\ 4 & 4 \end{bmatrix}$$

The diagonal entries 4 and 4 are the variances of the two features. The off-diagonal value 4 is their covariance. It is positive because both features increase together.

Step 4: Find eigenvalues and eigenvectors

Solve

$$Cv = \lambda v.$$

For this matrix the eigenvalues are

$$\lambda_1 = 8, \quad \lambda_2 = 0.$$

A unit eigenvector corresponding to the largest eigenvalue 8 is

$$v_1 = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \end{bmatrix} \approx \begin{bmatrix} 0.7071 \\ 0.7071 \end{bmatrix}.$$

This is the **first principal component direction**. The largest eigenvalue is selected because it represents the direction containing the greatest variance.

Step 5: Project the data onto the first principal component

Multiply the centered matrix by v_1 :

$$Z = X_c v_1.$$

Thus

$$Z = \begin{bmatrix} -2 & -2 \\ 0 & 0 \\ 2 & 2 \end{bmatrix} \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} -2\sqrt{2} \\ 0 \\ 2\sqrt{2} \end{bmatrix} \approx \begin{bmatrix} -2.828 \\ 0 \\ 2.828 \end{bmatrix}.$$

The original data had two coordinates per observation. After projection, each observation is represented by only one PCA score. This is the essential dimension-reduction idea.

PCA visual trace for the simple example

$$\begin{array}{ccccc}
 \underbrace{\begin{bmatrix} 2 & 1 \\ 4 & 3 \\ 6 & 5 \end{bmatrix}}_{\text{original data } X} & \longrightarrow & \underbrace{\begin{bmatrix} -2 & -2 \\ 0 & 0 \\ 2 & 2 \end{bmatrix}}_{\text{centered } X_c} & \longrightarrow & \underbrace{\begin{bmatrix} 4 & 4 \\ 4 & 4 \end{bmatrix}}_{\text{covariance } C} \\
 & & & & \\
 \longrightarrow \underbrace{\lambda_{\max} = 8, \quad v_1 = \frac{1}{\sqrt{2}}(1, 1)^T}_{\text{largest-variance direction}} & \longrightarrow & \underbrace{\begin{bmatrix} -2.828 \\ 0 \\ 2.828 \end{bmatrix}}_{\text{one-dimensional PCA scores}} .
 \end{array}$$

How the same steps appear in the Lab-9 code

For the actual problem, X contains 100 rows and 5 columns:

$$\underbrace{X}_{100 \times 5} .$$

The command

```
np.mean(X, axis=0)
```

computes the mean of each of the 5 columns. Hence

$$X_{\text{centered}} = X - \text{column means}.$$

The command

```
np.cov(X_centered, rowvar=False)
```

treats the **columns as variables/features** and produces a 5×5 covariance matrix. The option `rowvar=False` is important because the rows are observations and the columns are features.

Next,

```
eigvals, eigvecs = np.linalg.eig(cov)
```

returns 5 eigenvalues and 5 corresponding eigenvectors. The command

```
np.argmax(eigvals)
```

returns the position of the largest eigenvalue. Then

```
eigvecs[:, np.argmax(eigvals)]
```

selects the entire column corresponding to that eigenvalue; this column is the first principal component vector.

Finally,

```
projection = X_centered @ pc1
```

computes

$$\underbrace{X_c}_{100 \times 5} \underbrace{v_1}_{5 \times 1} = \underbrace{Z}_{100 \times 1} .$$

Thus the five original features are represented by one first-principal-component score for each of the 100 observations.

Python program

```
1 import numpy as np
2
3 X = np.random.rand(100, 5)
4
5 X_centered = X - np.mean(X, axis=0)
6
7 cov = np.cov(X_centered, rowvar=False)
8
9 eigvals, eigvecs = np.linalg.eig(cov)
10
11 pc1 = eigvecs[:, np.argmax(eigvals)]
12
13 projection = X_centered @ pc1
14
15 print(projection)
```

Visual Trace

$$X_{100 \times 5} \xrightarrow{\text{center columns}} X_{c, 100 \times 5} \xrightarrow{\text{covariance}} C_{5 \times 5} \xrightarrow{\text{eigen-decomposition}} (\lambda_i, v_i)$$
$$\xrightarrow{\text{largest } \lambda_i} v_1 \xrightarrow{X_c v_1} Z_{100 \times 1}.$$

The output `projection` is therefore a one-dimensional array containing 100 PCA scores.

Very important interpretation

PCA does **not** simply delete four columns. It constructs a new direction from all five original features. The first principal component is a weighted combination of those five features, and the weights are the entries of `pc1`. The largest eigenvalue tells us that this direction contains more variance than any other principal direction.

Program 9. Most Influential Node from a Dominant Eigenvector

Problem

Generate a 6×6 adjacency matrix, compute its eigenvalues and eigenvectors, and identify the node having the largest component in the dominant eigenvector.

Adjacency matrix

An adjacency matrix records the connections of a graph. For nodes 0, 1, 2, 3, 4, 5,

$$a_{ij} = 1$$

means that node i is connected to node j , whereas $a_{ij} = 0$ means that no such connection is present. The diagonal is normally zero for a graph without self-loops.

First draw the graph, then write the adjacency matrix

Consider the following easy six-node undirected graph.

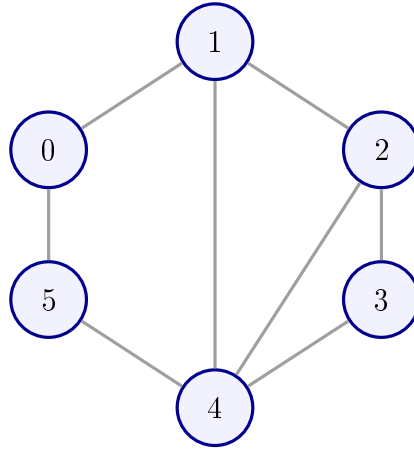


Figure: An illustrative six-node graph used to learn how an adjacency matrix is constructed.

Read one row at a time. Node 0 is connected to nodes 1 and 5, so row 0 is

$$[0, 1, 0, 0, 0, 1].$$

Node 1 is connected to nodes 0, 2 and 4, so row 1 is

$$[1, 0, 1, 0, 1, 0].$$

Continuing in the same manner gives

$$A = \begin{bmatrix} 0 & 1 & 0 & 0 & 0 & 1 \\ 1 & 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 1 & 0 & 1 \\ 1 & 0 & 0 & 0 & 1 & 0 \end{bmatrix}.$$

Because the example graph is undirected, A is symmetric: $a_{ij} = a_{ji}$.

A matrix-style visual check

The same information can be read as a 0–1 grid. A 1 marks an edge and a 0 marks no edge.

		column node j					
		0	1	2	3	4	5
row node i	0	0	1	0	0	0	1
	1	1	0	1	0	1	0
	2	0	1	0	1	1	0
	3	0	0	1	0	1	0
	4	0	1	1	1	0	1
	5	1	0	0	0	1	0

Why does the dominant eigenvector measure influence?

If

$$Av = \lambda v,$$

the entry v_i gives a score for node i . In eigenvector centrality, a node has high influence not merely because it has many neighbours, but because it is connected to neighbours that themselves have high influence. The eigenvector corresponding to the dominant eigenvalue gives the principal influence pattern.

If

$$v = (v_0, v_1, \dots, v_5)^T,$$

the program selects

$$\boxed{\arg \max_i |v_i|}.$$

Python program

```
1 import numpy as np
2
3 A = np.random.randint(0, 2, (6, 6))
4
5 eigvals, eigvecs = np.linalg.eig(A)
6
7 dominant = eigvecs[:, np.argmax(eigvals)]
8
9 node = max((i for i in range(len(dominant))),
10           key=lambda i: abs(dominant[i]))
11
12 print("Most influential node:", node)
```

Visual Trace

The code does the following:

random 6×6 matrix A $\xrightarrow{\text{np.linalg.eig}}$ eigenvalues and eigenvectors

$\xrightarrow{\text{np.argmax(eigvals)}}$ selected dominant eigenvector $\xrightarrow{\max |v_i|}$ most influential node index.

For example, if the selected eigenvector were

$$v = (0.20, 0.31, 0.42, 0.18, 0.71, 0.39)^T,$$

then the largest absolute entry is 0.71, occurring at index 4. Therefore the program would print

```
1 Most influential node: 4
```

Program 10. Quick Revision: What Each Program Teaches

No.	Main concept	Key mathematical/computational idea
1	Normalization	$z = (x - \mu)/\sigma$; transformed mean 0, standard deviation 1
2	Matrix filtering	Flatten, remove zeros, sum x^2
3	Monte-Carlo π	Area ratio $N_{\text{inside}}/N \approx \pi/4$
4	Matrix stability	Every eigenvalue must satisfy $ \lambda < 1$
5	Sliding windows	Move a length-10 window through 200 observations; <code>np.cov(w)</code>
7	Descriptive statistics	Mean, variance and median
8	Correlation	Select vector pairs with Pearson $r > 0.9$
9	PCA	Covariance matrix $\rightarrow$ largest eigenvalue/eigenvector $\rightarrow$ projection
10	Graph influence	Dominant eigenvector component identifies an influential node

Generator expression pattern

A generator expression generally has the form

(expression for item in iterable if condition).

Unlike a list comprehension, it does not immediately store every generated element in a list; elements are produced as they are requested.

Most important NumPy functions in these nine programs

Function	Purpose
<code>np.random.rand</code>	Uniform random values in $[0, 1)$
<code>np.random.randn</code>	Standard-normal random values
<code>np.random.randint</code>	Random integers
<code>np.fromiter</code>	Build an array from an iterable/generator
<code>np.mean</code> , <code>np.std</code> , <code>np.var</code> , <code>np.median</code>	Basic descriptive statistics
<code>np.cov</code>	Covariance/variance computation
<code>np.corrcoef</code>	Correlation coefficient matrix
<code>np.linalg.eigvals</code>	Eigenvalues only
<code>np.linalg.eig</code>	Eigenvalues and eigenvectors
<code>np.argmax</code>	Index of maximum entry
<code>np.abs</code>	Magnitude/absolute value
<code>np.all</code>	True only when all Boolean tests are True